Лабораторная работа №4

Вероятностная верификация модели турникета метро в PRISM

Касимов Владислав Андреевич

2026

1. Постановка задачи
2. Вероятностная модель
3. Свойства (PCTL и LTL внутри `P`)
4. Результаты верификации
5. Нетривиальность посылок
6. Количественный анализ
7. Анализ невыполненного свойства PR4
8. Сравнение nuXmv и PRISM
Приложение А. Модель `turnstile.pm`
Приложение Б. Свойства `turnstile.props`

1. Постановка задачи

В систему из ЛР1 (турникет в метро, вариант 2) нужно ввести вероятностные элементы: вероятности действий среды и сбоев датчиков. Получившуюся модель надо описать на языке PRISM, сформулировать три свойства на вероятностной темпоральной логике, проверить их в PRISM и сравнить подходы nuXmv и PRISM. Требования к системе и к свойствам те же, что в ЛР1.

2. Вероятностная модель

Тип модели и семантика

Модель — дискретная цепь Маркова (**DTMC**). Все восемь модулей синхронизированы по общему действию `[tick]`, и на каждом такте каждый модуль делает ровно один шаг. Это та же синхронная семантика, что в nuXmv: модуль видит значения других модулей на текущем такте, а реагирует на следующем. Структура модулей и переходов повторяет `turnstile.smv` один к одному.

Почему DTMC, а не MDP: в синхронной модели нет недетерминизма планировщика, потому что все модули шагают одновременно. Недетерминизм nuXmv был только в выборе среды, и в PRISM он целиком заменён вероятностями. В MDP пришлось бы искать минимум и максимум по стратегиям, хотя здесь выбирать некому.

Вероятностные параметры

Параметр	Значение	Смысл
`p_arr`	0.3	за такт к свободному турникету подходит пассажир
`q_card_ok` / `q_card_empty`	0.55 / 0.10	доли пассажиров с действующей и пустой картой
`q_token_ok` / `q_token_bad`	0.30 / 0.05	доли с правильным и посторонним жетоном
`p_err`	0.03	**сбой датчика**: валидатор не прочитал действующую карту (ложный отказ)
`p_fa`	0.02	**сбой датчика**: приёмник принял посторонний жетон (ложный приём)
`p_int`	0.05	за такт появляется безбилетник
`p_sneak` / `p_stay`	0.3 / 0.5	при открытых створках безбилетник проскакивает или продолжает ждать (иначе уходит)

Константы `p_err` и `p_sneak` передаются через `-const` (в `run.sh`: 0.03 и 0.3).

Пример: валидатор со сбоями

```
module card_reader
  cr : [0..2] init NONE;
  [tick] cr!=NONE & c=IDLE                -> (cr'=NONE);
  [tick] card_presented & m=CARD_OK       -> (1-p_err) : (cr'=OK) + p_err : (cr'=FAIL);
```

```
  [tick] card_presented & m=CARD_EMPTY     -> (cr'=FAIL);
  [tick] !(cr!=NONE & c=IDLE) & !card_presented -> true;
endmodule
```

В синхронизированной DTMC охранники команд одного модуля должны покрывать все состояния, иначе по `[tick]` возникнет тупик. Поэтому в каждом модуле есть явная команда `-> true` для «ничего не происходит».

Если отключить сбои датчиков (`p_err = p_fa = 0`), получается 108 достижимых состояний, как в nuXmv. Со сбоями их 133 (408 переходов).

Полный текст модели приведён в приложении А.

3. Свойства (PCTL и LTL внутри P)

№	Формула PRISM	Логика и структура	Смысл
PR1	`filter(forall, P>=1 [ g=0 U l=2 ], "rejected")`	PCTL, Until с вероятностью 1	аналог P1: во всех состояниях отказа створки п.н. закрыты, пока не загорится красный
PR2	`filter(forall, P>=0.95 [ l!=2 U<=10 u=3 ], "valid_card_presented")`	PCTL, **ограниченный** Until	аналог P2 с учётом сбоев: пассажир с действующей картой проходит за ≤10 тактов, не увидев красного, с вероятностью ≥ 0.95
PR3	`P>=1 [ G F (c=0 & g=0) ]`	LTL внутри `P`, живость	аналог L1: контроллер почти наверное бесконечно часто возвращается в исходное состояние
PR4	`P<=0.01 [ F<=100 "free_ride" ]`	PCTL, ограниченный Eventually	аналог P4: безбилетный проход за 100 тактов не чаще чем в 1% случаев (**ожидается нарушение**)

`filter(forall, φ, "rejected")` проверяет `φ` во всех *достижимых* состояниях, где выполнена посылка. Это вероятностный аналог `AG(p → φ)`. Свойства PCTL по умолчанию проверяются только в начальном состоянии.

Почему P2 переформулировано. Со сбоями `G(paid → F walk)` по-прежнему выполняется с вероятностью 1. Но действующую карту теперь могут ложно отклонить, и пассажир уйдёт, не пройдя. Поэтому интересно уже не «пройдёт ли», а «с какой вероятностью и за сколько тактов». Посылкой выбрано событие «предъявлена действующая карта», а не «датчик подтвердил оплату», чтобы сбой датчика вошёл в вероятность.

4. Результаты верификации

Прогон: `bash lab4-prism/run.sh`. PRISM 4.11, движок Hybrid (по умолчанию). Построение модели заняло 0.03 с, проверка каждого свойства 0.001–0.02 с, весь прогон 0.42 с вместе со стартом JVM.

Свойство	Результат	Численное значение
PR1	**true**	`rejected` выполнено в 21 состоянии, во всех P = 1
PR2	**true**	min P = **0.97** по 11 состояниям `valid_card_presented`
PR3	**true**	одна BSCC, совпадающая со всем пространством состояний
PR4	**false**	P(F≤100 free_ride) = **0.782 ≫** 0.01

Выполняются три свойства из четырёх.

Почему PR2 = 0.97 = 1 − p_err. Если карта прочитана, остальной путь детерминирован: `granted → open → walk`. Единственный источник отказа — сбой валидатора. Проход не зависит от безбилетника: тот проскакивает *вслед* за пассажиром и пройти ему не мешает.

Почему PR3 выполняется без справедливости. Модель конечна, и все 133 состояния образуют одну неразложимую компоненту (BSCC), в которой есть `c=0 & g=0`. В конечной DTMC путь с вероятностью 1 попадает в BSCC и посещает каждое её состояние бесконечно часто. Вероятность 1 здесь играет роль справедливости: «несправедливые» пути, где, например, пассажир вечно не приходит, существуют, но их суммарная мера равна 0.

5. Нетривиальность посылок

В вероятностной модели «посылка выполняется бесконечно часто» проверяется напрямую, через `P>=1 [ G F p ]`. Это сильнее, чем `AG EF p` в nuXmv: посылка повторяется не на каком-то пути, а почти на всех.

Проверка	Формула	Для	Результат
V1	`P>=1 [ G F "rejected" ]`	PR1	**true**
V2	`P>=1 [ G F "paid" ]`	P2	**true**
V3	`P>=1 [ G F "valid_card_presented" ]`	PR2	**true**

Посылка PR4 (`free_ride`) тоже встречается бесконечно часто: стационарная доля тактов с безбилетником в проходе равна 0.0148 > 0.

6. Количественный анализ

Ожидаемые значения за 1000 тактов (награды `C<=1000`)

Величина	Значение
оплаченных проходов (`passengers`)	**98.8**
безбилетных проходов (`free_rides`)	**14.7**
ложных отказов действующей карты (`false_rejects`, тактов с красным)	1.96

Величина	Значение
доля времени с открытыми створками, `S=? [ g=1 ]`	**0.397**
доля тактов с безбилетником, `S=? [ "free_ride" ]`	0.0148

Примерно на 7 оплаченных проходов приходится один безбилетный (14.7 / 98.8 ≈ 15%). Это количественная оценка уязвимости, которую nuXmv показал как один контрпример.

Время прохода

`filter(min, P=? [ l!=2 U<=K u=3 ], "valid_card_presented")` для K = 0…15:

K	0–4	5–15
min P	0	0.97

Функция ступенчатая. Конвейер такой: предъявление → датчик → контроллер → створки → проход. Синхронный конвейер детерминирован, поэтому в лучшем случае проход занимает 4 такта, а в худшем 5 (когда пассажир подошёл, пока контроллер ещё закрывал створки за предыдущим). Минимум по состояниям при K = 4 равен 0, максимум 0.97. После K = 5 вероятность уже не растёт: ложно отклонённый пассажир уходит и в этом заходе не пройдёт.

7. Анализ невыполненного свойства PR4

PRISM не строит контрпример для вероятностного свойства. Нарушение `P<=0.01` означает, что *мера* множества плохих путей больше порога, и одним путём это не покажешь. Типичный путь получен симулятором (`prism -simpath`), переменные `u m i cr tr beam c g l`:

Такт	u	m	i	cr	c	g	l	Пояснение
39	approach	card_ok	**behind**	none	closing	open	green	подходит пассажир, за ним безбилетник
40	wait	card_ok	behind	**ok**	closing	closed	off	валидатор принял карту
41	wait	card_ok	behind	ok	idle	closed	off	контроллер освободился
42	wait	card_ok	behind	none	**granted**	closed	off	проход разрешён
43	wait	card_ok	behind	none	granted	**open**	green	створки открыты
44	**walk**	card_ok	**sneak**	none	granted	open	green	**оба в проходе: безбилетный проход**
45	away	none	none	none	granted	open	green	`beam = true`: датчик видит одно перекрытие

Такт	u	m	i	cr	c	g	l	Пояснение
46	away	none	none	none	closing	open	green	контроллер закрывает створки как после одного прохода

Сценарий тот же, что во всех предыдущих работах. Обоснование, почему исправление слишком сложное, приведено в ЛР1, раздел 7.

8. Сравнение nuXmv и PRISM

Аспект	nuXmv (ЛР1)	PRISM (ЛР4)
Модель	конечный автомат, недетерминизм	DTMC: недетерминизм среды заменён вероятностями (доступны также MDP, CTMC, PTA)
Композиция	синхронная, модули	синхронизация по общему действию `[tick]`; без общего действия было бы чередование
Размер	108 состояний	108 без сбоев, 133 со сбоями датчиков
Логика	CTL, LTL: «выполняется / нет»	PCTL, LTL внутри `P`, награды (`R`), стационарные (`S`): **количественные ответы**
Вопрос	«возможно ли нарушение?»	«насколько вероятно нарушение? сколько раз за 1000 тактов?»
Справедливость	`FAIRNESS` при необходимости	заменена вероятностью 1: несправедливые пути имеют меру 0
Нетривиальность	`AG EF p`	`P>=1 [ G F p ]`, строже: почти на всех путях
Контрпример	трасса (лассо)	нет; для наглядности используется симулятор
Время-ограниченные свойства	только через счётчики в модели	встроенные `U<=k`, `F<=k`, `C<=k`
Алгоритм	BDD, неподвижные точки	BDD + численные методы (итерации Якоби/Гаусса — Зейделя), анализ BSCC
Время	0.02 с	0.42 с (вместе со стартом JVM)

Выводы о методах.

- nuXmv отвечает на вопрос «*может ли* случиться плохое», и одного контрпримера достаточно, чтобы свойство было ложным. PRISM отвечает на вопрос «*как часто*». Для P4 это даёт инженерно полезную информацию: ~15 безбилетников на 100 оплаченных проходов.
- Вероятностная модель позволяет моделировать сбои датчиков, и свойства формулируются с допуском (PR2 ≥ 0.95). В nuXmv сбой датчика можно ввести только как недетерминированную

альтернативу. Тогда свойство «пассажир с действующей картой пройдёт» сразу стало бы ложным, и оценить, насколько редко это случается, было бы нельзя.

- С другой стороны, для качественных свойств (PR1, PR3) PRISM лишь подтверждает то же, что nuXmv, без трассы при нарушении. Для поиска логических ошибок nuXmv удобнее, а PRISM нужен для оценки надёжности и производительности.

Приложение А. Модель `turnstile.pm`

```
// ======================================================================
// Лабораторная работа №4. Турникет в метро (вариант 2), PRISM, DTMC.
// Все модули синхронизируются по общему действию [tick], поэтому
// семантика совпадает с синхронной моделью nuXmv из ЛР1: на каждом
// такте все компоненты делают шаг одновременно. Недетерминированные
// выборы среды заменены вероятностями, добавлены сбои датчиков.
// ======================================================================
dtmc

// ---------- кодировка значений ----------
const int AWAY=0;  const int APPROACH=1; const int WAIT=2; const int WALK=3;
const int NONE=0;  const int CARD_OK=1;  const int CARD_EMPTY=2; const int TOKEN_OK=3; const
int TOKEN_BAD=4;
const int BEHIND=1; const int SNEAK=2;
const int OK=1;    const int FAIL=2;
const int IDLE=0;  const int GRANTED=1;  const int DENIED=2; const int CLOSING=3;
const int CLOSED=0; const int OPEN=1;
const int OFF=0;   const int GREEN=1;    const int RED=2;

// ---------- вероятностные параметры ----------
const double p_arr = 0.3;      // за такт к свободному турникету подходит пассажир
const double q_card_ok    = 0.55;  // распределение носителей
const double q_card_empty = 0.10;
const double q_token_ok   = 0.30;
const double q_token_bad  = 1 - q_card_ok - q_card_empty - q_token_ok;
const double p_err;            // (0.03, задаётся в run.sh) валидатор не прочитал исправную
карту (ложный отказ)
const double p_fa  = 0.02;     // приёмник принял посторонний жетон (ложный приём)
const double p_int = 0.05;     // за такт появляется безбилетник
const double p_sneak;          // (0.3, задаётся в run.sh) безбилетник проскакивает при
открытых створках
const double p_stay  = 0.5;    // ... или продолжает ждать

// ---------- пользователь / среда ----------
module passenger
  u : [0..3] init AWAY;
  m : [0..4] init NONE;
  [tick] u=AWAY -> (1-p_arr) : true
                 + p_arr*q_card_ok    : (u'=APPROACH)&(m'=CARD_OK)
                 + p_arr*q_card_empty : (u'=APPROACH)&(m'=CARD_EMPTY)
                 + p_arr*q_token_ok   : (u'=APPROACH)&(m'=TOKEN_OK)
                 + p_arr*q_token_bad  : (u'=APPROACH)&(m'=TOKEN_BAD);
  [tick] u=APPROACH                      -> (u'=WAIT);
  [tick] u=WAIT & g=OPEN                 -> (u'=WALK);
  [tick] u=WAIT & g=CLOSED & l=RED       -> (u'=AWAY)&(m'=NONE);
  [tick] u=WAIT & g=CLOSED & l!=RED      -> true;
  [tick] u=WALK                          -> (u'=AWAY)&(m'=NONE);
endmodule

module intruder
  i : [0..2] init NONE;
  [tick] i=NONE             -> (1-p_int) : true + p_int : (i'=BEHIND);
  [tick] i=BEHIND & g=OPEN  -> p_sneak : (i'=SNEAK) + p_stay : true + (1-p_sneak-p_stay) :
(i'=NONE);
  [tick] i=BEHIND & g=CLOSED-> 0.8 : true + 0.2 : (i'=NONE);
  [tick] i=SNEAK            -> (i'=NONE);
```

```
endmodule

// ---------- датчики ----------
formula card_presented  = cr=NONE & u=APPROACH & (m=CARD_OK  | m=CARD_EMPTY);
formula token_presented = tr=NONE & u=APPROACH & (m=TOKEN_OK | m=TOKEN_BAD);

module card_reader
  cr : [0..2] init NONE;
  [tick] cr!=NONE & c=IDLE                -> (cr'=NONE);
  [tick] card_presented & m=CARD_OK       -> (1-p_err) : (cr'=OK) + p_err : (cr'=FAIL);
  [tick] card_presented & m=CARD_EMPTY    -> (cr'=FAIL);
  [tick] !(cr!=NONE & c=IDLE) & !card_presented -> true;
endmodule

module token_acceptor
  tr : [0..2] init NONE;
  [tick] tr!=NONE & c=IDLE                -> (tr'=NONE);
  [tick] token_presented & m=TOKEN_OK     -> (tr'=OK);
  [tick] token_presented & m=TOKEN_BAD    -> p_fa : (tr'=OK) + (1-p_fa) : (tr'=FAIL);
  [tick] !(tr!=NONE & c=IDLE) & !token_presented -> true;
endmodule

module pass_sensor
  beam : bool init false;
  [tick] true -> (beam' = (u=WALK | i=SNEAK));
endmodule

// ---------- управляющий алгоритм ----------
module controller
  c : [0..3] init IDLE;
  [tick] c=IDLE & (cr=OK | tr=OK)                                -> (c'=GRANTED);
  [tick] c=IDLE & !(cr=OK | tr=OK) & (cr=FAIL | tr=FAIL)         -> (c'=DENIED);
  [tick] c=IDLE & !(cr=OK | tr=OK) & !(cr=FAIL | tr=FAIL)        -> true;
  [tick] c=GRANTED & beam                                        -> (c'=CLOSING);
  [tick] c=GRANTED & !beam                                       -> true;
  [tick] c=DENIED                                                -> (c'=IDLE);
  [tick] c=CLOSING & g=CLOSED                                    -> (c'=IDLE);
  [tick] c=CLOSING & g=OPEN                                      -> true;
endmodule

// ---------- актуаторы ----------
module gate
  g : [0..1] init CLOSED;
  [tick] true -> (g' = (c=GRANTED ? OPEN : CLOSED));
endmodule

module lamp
  l : [0..2] init OFF;
  [tick] true -> (l' = (c=GRANTED ? GREEN : (c=DENIED ? RED : OFF)));
endmodule

// ---------- метки и награды ----------
label "paid"     = cr=OK   | tr=OK;
label "rejected" = cr=FAIL | tr=FAIL;
label "valid_card_presented" = u=APPROACH & m=CARD_OK;
label "free_ride" = i=SNEAK;

rewards "free_rides"  i=SNEAK : 1; endrewards   // безбилетных проходов
rewards "passengers"  u=WALK  : 1; endrewards   // оплаченных проходов
rewards "false_rejects" u=WAIT & l=RED & (m=CARD_OK) : 1; endrewards
```

Приложение Б. Свойства `turnstile.props`

```
// ===== PR1 (PCTL, Until с вероятностью 1, вложение через filter) =====
// Во всех достижимых состояниях, где датчик отклонил оплату, с вероятностью 1
// створки остаются закрытыми, пока не загорится красный.
"PR1": filter(forall, P>=1 [ g=0 U l=2 ], "rejected")

// ===== PR2 (PCTL, ограниченный Until) =====
// Пассажир с действующей картой проходит в течение 10 тактов, не увидев
// красного сигнала, с вероятностью не менее 0.95 (худший случай по состояниям).
"PR2": filter(forall, P>=0.95 [ l!=2 U<=10 u=3 ], "valid_card_presented")
"PR2_value": filter(min, P=? [ l!=2 U<=10 u=3 ], "valid_card_presented")

// ===== PR3 (LTL внутри P: живость с вероятностью 1) =====
// Почти наверное контроллер бесконечно часто возвращается в исходное состояние.
"PR3": P>=1 [ G F (c=0 & g=0) ]

// ===== PR4 (ОЖИДАЕТСЯ НАРУШЕНИЕ) =====
// Вероятность хотя бы одного безбилетного прохода за 100 тактов не выше 1%.
"PR4": P<=0.01 [ F<=100 "free_ride" ]
"PR4_value": P=? [ F<=100 "free_ride" ]

// ===== Количественный анализ =====
"free_rides_1000":  R{"free_rides"}=? [ C<=1000 ]
"passengers_1000":  R{"passengers"}=? [ C<=1000 ]
"false_rej_1000":   R{"false_rejects"}=? [ C<=1000 ]
"free_ride_share":  S=? [ "free_ride" ]
"gate_open_share":  S=? [ g=1 ]

// ===== Нетривиальность посылок: выполняются бесконечно часто п.н. =====
"V1": P>=1 [ G F "rejected" ]
"V2": P>=1 [ G F "paid" ]
"V3": P>=1 [ G F "valid_card_presented" ]
```